

S DES 系统开发手册

本手册描述当前提交版本的模块组织、算法流程、必要公共接口、数据约定与异常行为，供维护、测试和复用代码时使用。算法核心不依赖 PyQt6，可以在命令行脚本中独立调用。

1 模块和调用关系

```
main.py
└─ gui/main_window.py
    ├─ BasicPage      → core/cipher.py 与 key_schedule.py
    ├─ TextPage       → core/text_cipher.py → cipher.py
    ├─ BruteForcePage → FunctionWorker → core/brute_force.py
    └─ AnalysisPage   → FunctionWorker → core/collision_analysis.py
核心底层: constants.py 与 bit_utils.py
```

文件	职责
core/constants.py	置换表和两个 S 盒常量
core/bit_utils.py	二进制校验、置换、异或、循环移位及整数转换
core/key_schedule.py	K1 和 K2 生成及密钥扩展轨迹
core/cipher.py	轮函数、半部交换、分组加解密及轨迹
core/text_cipher.py	ASCII 编码、逐字节加解密、HEX 转换
core/brute_force.py	全密钥枚举、候选筛选、进度与计时
core/collision_analysis.py	固定明文和完整明文空间统计
gui/workers.py	QRunnable 后台任务及完成、进度、错误信号
gui/*_page.py	收集输入、调用核心、显示结果和异常

2 数据与算法约定

所有分组和密钥均用保留前导零的字符串表示。分组 8 位，原始密钥 10 位，子密钥 8 位。核心校验先去首尾空白，再检查长度，最后检查字符只能为 0 或 1。置换表使用从 1 开始的位置。

2.1 常量

常量	值
P10	3, 5, 2, 7, 4, 10, 1, 9, 8, 6
P8	6, 3, 7, 4, 8, 5, 10, 9
IP	2, 6, 3, 1, 4, 8, 5, 7
IP_INV	4, 1, 3, 5, 7, 2, 8, 6
EP	4, 1, 2, 3, 2, 3, 4, 1

常量	值
SP	2, 4, 3, 1

SBOX1	SBOX2
1 0 3 2	0 1 2 3
3 2 1 0	2 3 1 0
0 2 1 3	3 0 1 2
3 1 0 2	2 1 0 3

本节常量采用作业第 2 节给定的参数，与 `core/constants.py` 一致。SBox2 按作业指定数值实现。作业链接：<https://shimo.im/docs/m5kvd1MaKvcENy3X/>

2.2 密钥扩展

当前代码将 P10 后的结果分成 L、R 两段 5 位数据。K1 由 L 和 R 各循环左移 1 位、拼接后经过 P8 得到；K2 由原来的 L 和 R 各循环左移 2 位、拼接后经过 P8 得到。两个移位分别以 P10 输出为起点。

```
P10(K) = L || R
K1 = P8(ROL(L, 1) || ROL(R, 1))
K2 = P8(ROL(L, 2) || ROL(R, 2))
```

对 $K=1010000010$ ，K1 为 10100100，K2 为 10010010。跨组比较前须统一这一解释及 S 盒值，否则使用其他实现变体会产生不同密文。

2.3 两轮加解密

```
加密: IP → fk(K1) → SW → fk(K2) → IP_INV
解密: IP → fk(K2) → SW → fk(K1) → IP_INV
fk(L || R, Ki) = (L XOR F(R, Ki)) || R
F(R, Ki) = SP(SBOX1(A) || SBOX2(B))
A || B = EP(R) XOR Ki
```

S 盒输入为 4 位 $b_1 b_2 b_3 b_4$ ，行索引是 $b_1 b_4$ ，列索引是 $b_2 b_3$ ，均从 0 开始；输出补足 2 位。SW 交换左右 4 位。解密复用同一流程，只反转子密钥顺序。

3 公共接口

3.1 密钥扩展接口

模块 `core.key_schedule`:

```
generate_subkeys(key: str) -> tuple[str, str]
generate_subkeys_with_trace(key: str) -> tuple[str, str, dict]
```

输入 10 位密钥，返回 (K1, K2)；轨迹版本额外返回含 `original_key`、`p10`、`p10_left`、`p10_right`、`left_shift_1`、`left_shift_2`、`k1`、`k2` 的字典。非法输入抛出 `ValueError`。

3.2 分组加解密接口

模块 `core.cipher`:

```
encrypt_block(plaintext: str, key: str) -> str
decrypt_block(ciphertext: str, key: str) -> str
encrypt_block_with_trace(plaintext: str, key: str) -> tuple[str, dict]
decrypt_block_with_trace(ciphertext: str, key: str) -> tuple[str, dict]
```

数据 8 位，密钥 10 位。普通版本返回 8 位结果；轨迹版本返回（结果，轨迹字典）。不符合长度或字符要求时抛出 `ValueError`。

顶层轨迹字段为 `input`、`key_schedule`、`ip`、`round1_key`、`round1`、`sw`、`round2_key`、`round2`、`ip_inv`。每轮记录输入、左右半部、轮函数 `f`、异或后左半部和输出；`f` 包含扩展、异或、S 盒输入输出和 SP 结果。界面通过 `gui.basic_page.format_trace` 将其转成可读文字。

3.3 字节与 ASCII 接口

模块 `core.text_cipher`:

```
encrypt_bytes(data: bytes, key: str) -> bytes
decrypt_bytes(data: bytes, key: str) -> bytes
encrypt_text_ascii(text: str, key: str) -> bytes
decrypt_text_ascii(ciphertext: bytes, key: str) -> str
bytes_to_hex(data: bytes) -> str
bytes_to_binary(data: bytes) -> str
hex_to_bytes(text: str) -> bytes
```

字节接口逐个字节调用分组接口，输入输出字节数相同，无填充。ASCII 加密先执行 `text.encode('ascii')`；解密后执行 ASCII 解码，不可编码或不可解码时转换为 `ValueError`。

HEX 输出为大写两位字节、以空格分隔；二进制输出为 8 位字节、以空格分隔。HEX 解析移除所有空白，要求偶数个十六进制数字，格式错误抛出 `ValueError`，空 HEX 返回 `b''`。

字节 API 在循环前先校验 10 位密钥，因此空数据也要求合法密钥；合法密钥下空字节返回 `b''`，ASCII 解密返回空字符串。当前 API 无消息完整性验证，错误密钥不一定导致异常。

3.4 暴力破解接口

模块 `core.brute_force`:

```
brute_force(
    known_pairs: Iterable[tuple[str, str]],
    progress_callback: Callable[[int], None] | None = None,
) -> BruteForceResult
```

`known_pairs` 至少一个元素，每个元素为 8 位明文、8 位密文二元组；所有数据应来自相同原密钥。空列表或无效分组抛出 `ValueError`。每个密钥必须同时满足所有明密文对，程序不在首次命中时退出。

返回字段	类型	含义
<code>candidate_keys</code>	<code>list[str]</code>	所有匹配的 10 位密钥，按枚举顺序排列
<code>tested_keys</code>	<code>int</code>	完整搜索为 1024
<code>elapsed_seconds</code>	<code>float</code>	<code>perf_counter</code> 测得的秒数
<code>started_at</code>	<code>str</code>	带时区、毫秒精度的 ISO 开始时间
<code>finished_at</code>	<code>str</code>	同格式结束时间

回调接收 0 到 100 的整数百分比，循环中每检查 32 个密钥通知一次，结束时再次通知 100，调用者应允许重复 100。无匹配时返回空候选列表，不作为异常。输入规格化发生在计时前，GUI 显示和渲染不计入算法耗时。

3.5 碰撞分析接口

模块 `core.collision_analysis`:

```
analyze_plaintext(plaintext: str) -> PlaintextCollisionResult
analyze_all_plaintexts(
    progress_callback: Callable[[int], None] | None = None,
) -> AllPlaintextsSummary
```

单明文接口输入 8 位字符串，非法输入抛出 `ValueError`。返回字段：

字段	类型	含义
<code>plaintext</code>	<code>str</code>	被分析明文
<code>ciphertext_to_keys</code>	<code>dict[str, list[str]]</code>	每个实际产生的密文及对应密钥，含非碰撞组
<code>distinct_ciphertexts</code>	<code>int</code>	不同密文的数量
<code>collision_ciphertexts</code>	<code>int</code>	对应至少两个密钥的密文数量
<code>max_keys_for_one_ciphertext</code>	<code>int</code>	单个密文对应的最多密钥数

全空间返回 `total_plaintexts`、`total_encryptions`、`plaintexts_with_collisions`、`min_distinct_ciphertexts`、`max_distinct_ciphertexts`、`average_distinct_ciphertexts`、`global_max_keys_for_one_ciphertext`、`elapsed_seconds`、`per_plaintext`。

`per_plaintext` 是 256 条字典组成的列表，各条包含 `plaintext`、`distinct_ciphertexts`、`collision_ciphertexts`、`max_keys_for_one_ciphertext`。每处理 4 个明文通知进度，结束再次通知 100。

3.6 底层工具接口

模块 `core.bit_utils`:

接口	返回与约束
<code>validate_binary(value, length, label)</code>	返回去除首尾空白且通过校验的字符串；不合法抛 <code>ValueError</code>
<code>permute(bits, table)</code>	按 1 基表取位；空表返回空串；越界抛 <code>ValueError</code>
<code>xor_bits(left, right)</code>	等长二进制字符串异或；长度不等或非法字符抛 <code>ValueError</code>
<code>left_rotate(bits, amount)</code>	循环左移，位数取模；空串抛 <code>ValueError</code>
<code>int_to_bits(value, width)</code>	输出补零二进制串；超出 0 到 $2^{\text{width}-1}$ 抛 <code>ValueError</code>
<code>bits_to_int(bits)</code>	返回整数；空串或非法字符抛 <code>ValueError</code>

4 调用示例

在“5.2.1 源代码”目录运行：

```
from core.cipher import encrypt_block, decrypt_block
from core.text_cipher import encrypt_text_ascii, bytes_to_hex
from core.brute_force import brute_force

key = '1010000010'
ciphertext = encrypt_block('10101010', key)
assert ciphertext == '00001001'
assert decrypt_block(ciphertext, key) == '10101010'
print(bytes_to_hex(encrypt_text_ascii('Hello SDES!', key)))
result = brute_force(['10101010', '00001001'])
print(result.candidate_keys)
```

5 后台任务与错误处理

FunctionWorker(function, *args, supports_progress=False) 继承 QRunnable。启用进度时将 signals.progress.emit 作为 progress_callback 传给核心函数；完成发出 finished(object)，异常发出 error(str)。各页面用 QThreadPool.globalInstance().start(worker) 执行任务，通过信号在界面显示结果。

Basic 和 Text 捕获 ValueError 并显示警告。Basic 执行前清除输出及轨迹；Text 加密前清除密文和结果，解密时保留 HEX 输入并清除旧解密结果。核心检查完整二进制输入的长度和字符。破解和分析出错时清空结果并恢复按钮状态。后台任务用于保持界面响应，密钥搜索采用串行枚举。

6 密钥等价与碰撞

1010000010 与 1110000010 生成相同的 K1 和 K2，因此对所有明文产生相同密文。当前 P8 和移位组合没有使用原密钥的第 2 位：将该位翻转不会改变子密钥，故仅凭明密文无法区分这些等价密钥。

固定明文下 1024 个密钥映射到最多 256 种密文，按抽屉原理必然存在碰撞。当前实现全空间实测每个明文产生 254 种密文，每种实际出现的密文至少对应两个密钥。254 和最大 8 个密钥是本实现的实测统计，不是仅凭抽屉原理即可推出的通用数值。

7 维护与复核

核心模块复用位运算、密钥扩展和分组加解密接口，S 盒寻址、Feistel 结构、候选筛选及碰撞分组均提供注释。修改算法参数后，应运行测试确认结果。核心测试命令为 python tests/verify_results.py；Qt 控件测试命令为 python tests/verify_gui.py。

两种脚本默认在 verification/runs/下新建带时间戳的 JSON；也可用 --output 指定新文件，例如 python tests/verify_results.py --output verification/runs/my_check.json。已有同名文件会被拒绝覆盖；相对输出路径以运行命令的当前目录为准。

复现环境与测试范围

依赖范围见 requirements.txt，Windows 11、Python 3.12.14 下已验证的精确版本见 requirements-verified.txt。核心测试使用 Python 标准库；GUI 测试需要 PyQt6，采用 offscreen 模式验证控件行为与信号。文本接口使用 ASCII 编码，网络通信不属于本系统接口范围。